

[EJERCICIO](#)

[USUARIOS](#)

[GET BY ID](#)

[LIBROS](#)

[GET BY ID](#)

[EJEMPLARES](#)

[GET BY ID](#)

[PRÉSTAMOS](#)

[GET BY ID](#)

[GITHUB](#)

EJERCICIO

USUARIOS

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id", nullable = false)
private Integer id;

@Column(name = "dni", nullable = false, length = 15) 3 usages
@NotNull
@NotBlank
@Pattern(regexp = "\\d{8}[A-Z]$", message = "El formato del DNI no es el correcto")
private String dni;

@Column(name = "nombre", nullable = false, length = 100) 3 usages
@NotNull
@NotBlank
@Pattern(regexp = "[a-zA-Z0-9]+", message = "El nombre solo puede tener caracteres alfanuméricos")
@Max(100)
private String nombre;

@Column(name = "email", nullable = false, length = 100) 3 usages
@NotNull
@NotBlank
@Pattern(regexp = "[A-Za-z0-9]{1,50}@gmail.com", message = "El formato del email no es el correcto")
private String email;

@Column(name = "password", nullable = false) 3 usages
@NotNull
@NotBlank
@Pattern(regexp = "[a-zA-Z0-9]{4,12}+", message = "El formato de la contraseña es el incorrecto")
private String password;

@Lob 3 usages
@Column(name = "tipo", nullable = false)
@NotNull
@NotBlank
@Pattern(regexp = "(normal|administrador)", message = "El formato del tipo solo adminte administrador o normal")
private String tipo;

@Column(name = "penalizacion_hasta") 3 usages
@NotBlank
private LocalDate penalizacionHasta;
```

GET BY ID

```
//Obtener usuario por ID
@GetMapping("/{id}") @Adrian Polo +1
@Cacheable
public ResponseEntity<Optional<Usuario>> obtenerUsuarioPorId(@PathVariable Integer id) {
    try{
        Thread.sleep( millis: 3000);
        Optional<Usuario> usuario = usuarioService.obtenerUsuarioById(id);
        return new ResponseEntity<>(usuario, HttpStatus.OK);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

GET http://localhost:8080/api/usuarios/1 Send

Params Authorization Headers (6) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results 200 OK 3 ms 312 B

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "dni": "123456789A",
4   "nombre": "Juan Perez",
5   "email": "juan.perez@ejemplo.com",
6   "password": "password123",
7   "tipo": "normal",
8   "penalizacionHasta": null
9 }
```

LIBROS

```
@Id
@Column(name = "isbn", nullable = false, length = 20)
@NotNull
@NotBlank
@Pattern(regexp = "^(?:\\d{3}-\\d{1}-\\d{5}-\\d{3}-\\d{1}|\\d{13}$)",message = "El formato del ISBN es incorrecto")
private String isbn;

@Column(name = "titulo", nullable = false, length = 200) 2 usages
@NotNull
@NotBlank
@Max(200)
private String titulo;

@Column(name = "autor", nullable = false, length = 100) 2 usages
@NotNull
@NotBlank
@Max(100)
private String autor;
```

GET BY ID

```
// ✅ Obtener un libro por ISBN
@GetMapping("/{isbn}")  Adrian Polo +1
@Cacheable
public ResponseEntity<Optional<Libro>> obtenerLibroPorIsbn(@PathVariable String isbn) {
    try{
        Thread.sleep( millis: 3000);
        Optional<Libro> libro = libroService.obtenerLibroPorIsbn(isbn);
        return new ResponseEntity<>(libro, HttpStatus.OK);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

GET http://localhost:8080/api/libro/9781111111111 Send

Params Authorization Headers (5) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results

JSON Preview Visualize

```
1 {
2   "isbn": "9781111111111",
3   "titulo": "1984",
4   "autor": "George Orwell"
5 }
```

200 OK 3 ms 228 B

EJEMPLARES

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id", nullable = false)
private Integer id;

@ManyToOne(fetch = FetchType.EAGER, optional = false) 2 usages
@OnDelete(action = OnDeleteAction.CASCADE)
@JoinColumn(name = "isbn", nullable = false)
@NotNull
@NotBlank
private Libro libro;

@ColumnDefault("'Disponible'") 2 usages
@Lob
@Column(name = "estado")
@Pattern(regexp = "^(disponible|prestado|dañado)$", message = "El formato del estado so puede ser disponible, prestado o dañado")
@NotBlank
private String estado;
```

GET BY ID

```
// Obtener ejemplar by ID
@GetMapping("/{id}") @Adrian Polo +1
@Cacheable
public ResponseEntity<Optional<Ejemplar>> obtenerEjemplarByID(@PathVariable Integer id) {
    try{
        Thread.sleep( millis: 3000);
        Optional<Ejemplar> ejemplar = ejemplarService.obtenerEjemplarByID(id);
        return new ResponseEntity<>(ejemplar, HttpStatus.OK);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

http://localhost:8080/api/ejemplares/1

GET http://localhost:8080/api/ejemplares/1

Params Authorization Headers (6) Body Scripts Tests Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "estado": "Disponible",
4   "isbn": {
5     "isbn": "9781234567890",
6     "titulo": "El Quijote",
7     "autor": "Miguel de Cervantes"
8   }
9 }
```

200 OK 4 ms 278 B

PRÉSTAMOS

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "id", nullable = false)
private Integer id;

@ManyToOne(fetch = FetchType.LAZY, optional = false) 2 usages
@OnDelete(action = OnDeleteAction.CASCADE)
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
@JoinColumn(name = "usuario_id", nullable = false)
@NotNull
@NotBlank
private Usuario usuario;

@ManyToOne(fetch = FetchType.LAZY, optional = false) 2 usages
@OnDelete(action = OnDeleteAction.CASCADE)
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
@JoinColumn(name = "ejemplar_id", nullable = false)
@NotNull
@NotBlank
private Ejemplar ejemplar;

@Column(name = "fechaInicio", nullable = false) 2 usages
@NotBlank
private LocalDate fechaInicio;

@Column(name = "fechaDevolucion") 2 usages
@NotBlank
private LocalDate fechaDevolucion;
```


GET BY ID

```
// ✅ Obtener un libro por ID (posición en la lista)
@GetMapping("/{id}")  Adrian Polo +1
@Cacheable
public ResponseEntity<Optional<Prestamo>> obtenerPrestamoPorID(@PathVariable Integer id) {
    try{
        Thread.sleep( millis: 3000);
        Optional<Prestamo> prestamo = prestamoService.obtenerPrestamoPorID(id);
        return new ResponseEntity<>(prestamo, HttpStatus.OK);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

GET http://localhost:8080/api/prestamos/1 Send

Params Authorization Headers (0) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (5) Test Results

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "usuario": {
4     "id": 1,
5     "dni": "12345678A",
6     "nombre": "Juan Perez",
7     "email": "juan.perez@example.com",
8     "password": "password123",
9     "tipo": "normal",
10    "penalizacionHasta": null
11  },
12  "ejemplar": {
13    "id": 1,
14    "estado": "Disponible",
15    "isbn": {
16      "isbn": "9781234567890",
17      "titulo": "El Quijote",
18      "autor": "Miguel de Cervantes"
19    }
20  },
21  "fechaInicio": "2024-11-01",
22  "fechaDevolucion": null
23 }
```

GITHUB

<https://github.com/Polo03/SpringBoot/tree/CacheBiblioteca>